

Assignment: Access Control Models and Linux Permissions, SetUID

There are two parts in this assignment. Each is worth 50 points. Part 1 invites you to practice access control, linux permissions and ACM, ACL. Part 2 requires you to practice with compiling programs, setuid and environmental variables.

PART 1: Access Control Models and Linux Permissions:

Total Points: 50

Student Name: Duy Linh Hoang_____ Student ID:M50123840 _____

Learning Objectives

Upon completing this assignment, you will be able to:

- Interpret business requirements and translate them into specific access control permissions.
- Construct and populate an Access Control Matrix (ACM) for a multi-process, multi-object system.
- Derive Access Control Lists (ACLs) from an Access Control Matrix.
- Map an access control policy to Linux owner/group/others permissions using chmod, chown, and chgrp.
- Implement and verify file permissions on a Linux (Ubuntu) system using command-line tools.

Part 0: Business Case — NovaTech Solutions

NovaTech Solutions is a mid-size software company that develops a cloud-based analytics platform. The engineering team has recently restructured its deployment pipeline and needs to enforce strict access control over three critical files on their shared development server. The company's security officer has outlined the following requirements after a recent audit.

The System

The server hosts three files and runs three processes (services). The team is organized into two functional groups:

Groups:

- **appteam** — The application development group. Members: webapp and dataengine. These two services both interact with the application's configuration and analytics code.
- **ops** — The operations group. Members: deployer. This group is responsible for deployment and release management.

Files:

- **config.yaml** — The master configuration file for the platform. Contains database credentials, API keys, and feature flags. Both application services need to read this file at startup, but only the Deployer writes to it (to update deployment timestamps and environment settings after each release).

- **analytics.py** — The core analytics processing script. DataEngine is the primary owner of this script: it reads, modifies, and executes it to run data pipelines. WebApp only needs to read analytics.py to display available report types in its interface. The Deployer has no reason to access this file.
- **deploy.sh** — The deployment shell script. Pulls the latest code, runs tests, and pushes to production. Only the Deployer needs to read and execute this script. No other service should be able to read, modify, or execute it.

Processes (Services):

- **P1 — WebApp:** The front-end web application service. It serves the user interface and reads config.yaml at startup to load feature flags and API endpoints. It also reads analytics.py to discover which report types are available for users. It must never modify the configuration, the analytics script, or the deployment script.
- **P2 — DataEngine:** The back-end data processing engine. It reads config.yaml to obtain database connection strings. It is the primary developer of analytics.py: it reads, writes, and executes the script to run data processing pipelines and generate reports. It has no involvement with deployments and must not access deploy.sh in any way.
- **P3 — Deployer:** The automated deployment agent. It reads config.yaml to verify environment settings before deployment, and writes updated deployment timestamps back into config.yaml after each release. It reads and executes deploy.sh to push new releases to production. It must never access analytics.py.

Security Policy Summary

The security officer has stated: *"Each service must operate under the principle of least privilege. No process should have any permission that it does not strictly need to perform its designated function. Read access should be granted only where data consumption is required, write access only where data modification is part of the service's role, and execute access only where a script must be run as a program. Any permission not explicitly required must be denied."*

Part 1: Deriving Permissions (10 points)

1. Carefully read the business case above. For each combination of process and file, determine the required permissions (Read, Write, Execute, or None). Justify each decision in one to two sentences.
 - a) List the permissions for P1 (WebApp) on each of the three files, with justification.
 - config.yaml → Read
 - The WebApp should read the configuration settings like database or server settings at the startup. It should not have the permission to change the configuration file as that could result in security or stability issues.
 - analytics.py → Read
 - The WebApp has to read the analytics script only to identify the reports that are available. It is not required to have the permission to edit or run the file directly.
 - deploy.sh → None
 - The WebApp should not have any access to deployment scripts. Granting it permissions to even execute or view deployment scripts could result in serious security issues.

b) List the permissions for P2 (DataEngine) on each of the three files, with justification.

- config.yaml → Read
 - The DataEngine needs to read the configuration file to obtain connection information, but it should not modify it.
- analytics.py → Read, Write, Execute
 - Since the DataEngine developer actively updates and runs analytics code, it needs full permissions.
- deploy.sh → None
 - The DataEngine has no responsibility for deployment, so it should not access deployment scripts.

c) List the permissions for P3 (Deployer) on each of the three files, with justification.

- config.yaml → Read, Write
 - The Deployer needs to read system settings and also write deployment timestamps or updates after releases.
- analytics.py → None
 - The Deployer should not have access to analytics code because that is not part of its role and could create unnecessary risk.
- deploy.sh → Read, Execute
 - The Deployer must be able to read and execute the deployment script, but should not modify it, since deployment scripts should remain controlled.

Part 2: Access Control Matrix (10 points)

Using your answers from Part 1, fill in the Access Control Matrix below. In each cell, write the applicable permissions using the notation R (Read), W (Write), X (Execute), or — (None).

Subject \ Object	config.yaml	analytics.py	deploy.sh
P1 (WebApp)	R	R	---
P2 (DataEngine)	R	RWX	---
P3 (Deployer)	RW	---	RX

2. Explain in a short paragraph why an Access Control Matrix is useful for system administrators when auditing permissions, and describe one limitation of this representation.
 - a) Access Control Matrix forms an essential part of the system administration, as it presents in a clear and formal way the users who are authorized to access certain resources and the particular actions they can perform with these resources such as read, write or execute the files. Thus, that use the Access Control Matrix is a major advantage for the system administrators in auditing permissions, finding misconfigurations, and ensuring that the right access are in line with the security policies, such as the right of the least privilege. The disadvantage of the Access Control Matrix is that it may become very large and tough to keep or operate in real systems because of this mainly in the situation of a large number of users or files and the biggest part of the matrix is sometimes missed by the users.

Part 3: Access Control Lists (10 points)

Convert the Access Control Matrix from Part 2 into Access Control Lists. For each file (object), list which processes have which permissions.

Fill in the ACL table below. Use the format: ProcessName: {permissions}, separated by commas for multiple entries.

Object (File)	Access Control List Entries
config.yaml	WebApp : {R}, DataEngine: {R}, Deployer : {RW}
analytics.py	WebApp : {R}, DataEngine: {RWX}
deploy.sh	Deployer : {RW}

3. Compare the ACM and ACL representations by answering the following:
 - d) Which representation makes it easier to answer the question: "What can P2 access?" Explain why.
 - The Access Control Matrix is easier because we can look at the row for P2 and immediately see all permissions.
 - e) Which representation makes it easier to answer: "Who can access deploy.sh?" Explain why.
 - ACLs are easier because we just check the deploy.sh list and see which processes have permissions.
 - f) Describe a scenario where you might prefer a Capability List over an ACL. What is the key structural difference?
 - A capability list is useful when the system wants each process to carry a list of resources it can access, such as in sandboxed applications or mobile operating systems. The difference is ACL is stored with the object, while capability is stored with the subject.

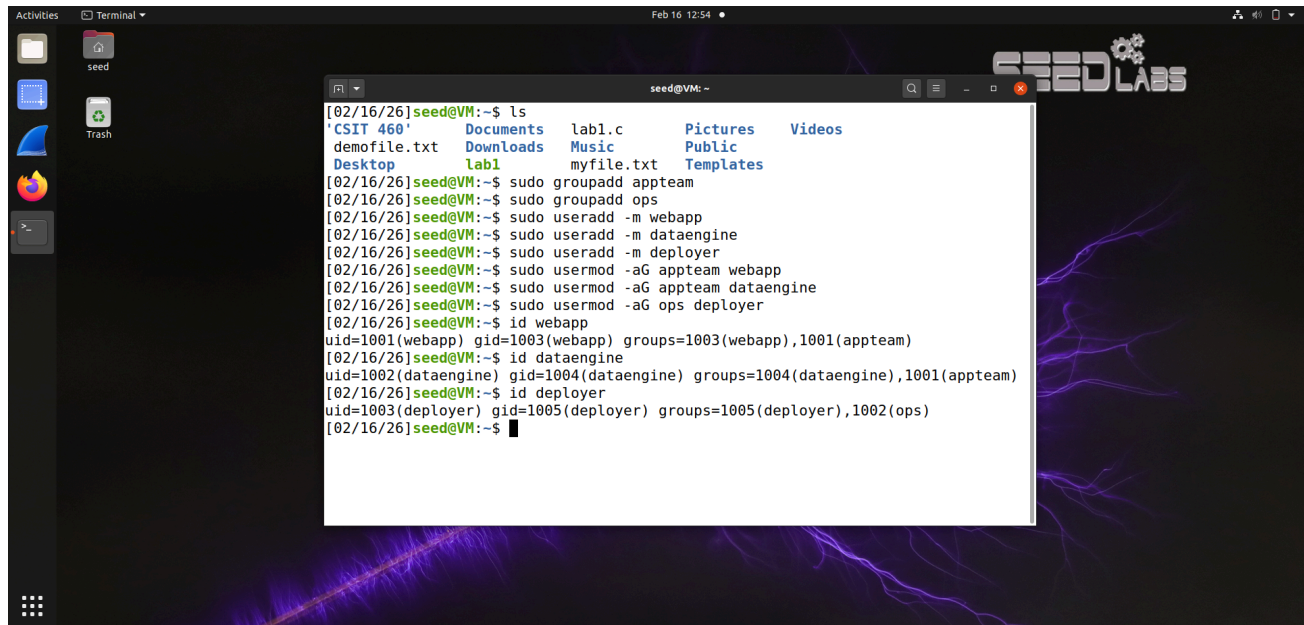
Part 4: Linux Implementation (20 points)

In this part, you will implement the access control policy from the NovaTech business case on an Ubuntu Linux system using standard file permissions (chmod, chown, chgrp). You will create

users, groups, files, and assign ownership and permissions that enforce the policy. Perform all tasks on an Ubuntu system (a VM, WSL, or cloud instance is acceptable). Record all commands you execute and capture terminal output as evidence.

Task 4.1 — Setup: Users and Groups (5 points)

Create three Linux user accounts to represent the three processes: webapp, dataengine, and deployer. Create two groups: appteam (containing webapp and dataengine) and ops (containing deployer). Provide the commands used and verify with the id command for each user.



```

[02/16/26]seed@VM:~$ ls
'CSIT 460'  Documents  lab1.c  Pictures  Videos
demofile.txt Downloads  Music   Public   Templates
Desktop     lab1      myfile.txt

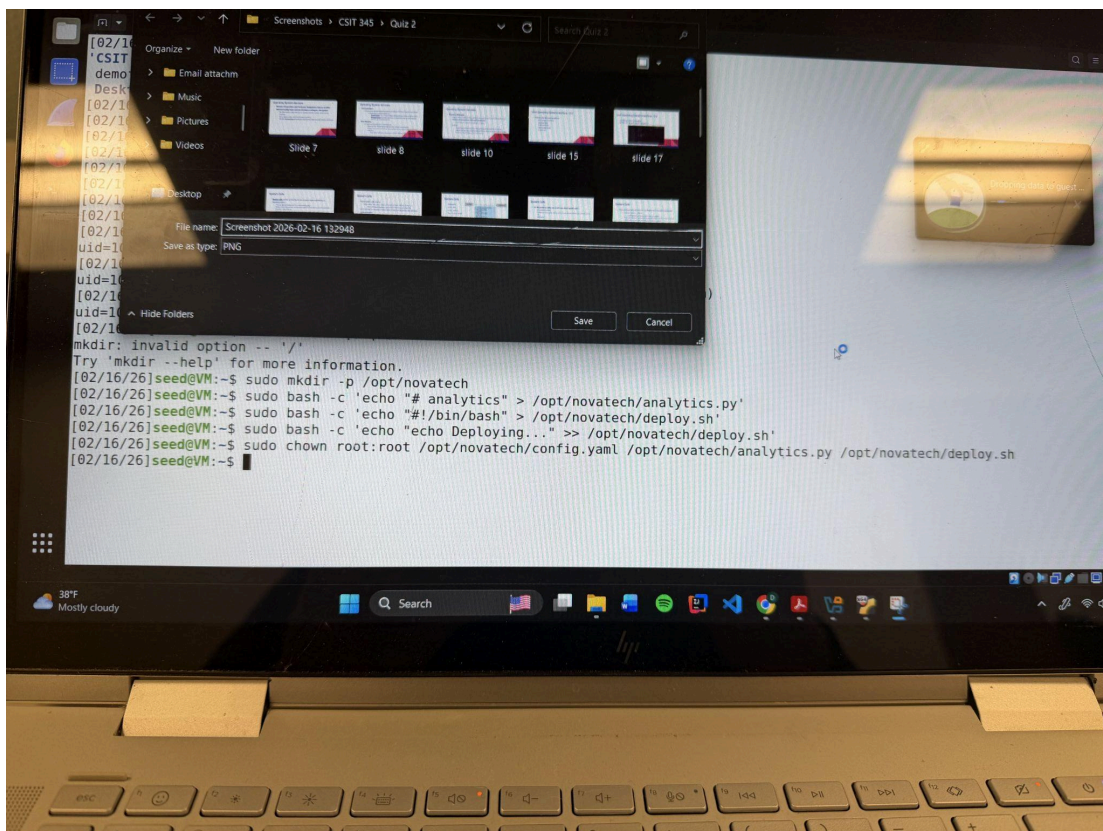
[02/16/26]seed@VM:~$ sudo groupadd appteam
[02/16/26]seed@VM:~$ sudo groupadd ops
[02/16/26]seed@VM:~$ sudo useradd -m webapp
[02/16/26]seed@VM:~$ sudo useradd -m dataengine
[02/16/26]seed@VM:~$ sudo useradd -m deployer
[02/16/26]seed@VM:~$ sudo usermod -aG appteam webapp
[02/16/26]seed@VM:~$ sudo usermod -aG appteam dataengine
[02/16/26]seed@VM:~$ sudo usermod -aG ops deployer
[02/16/26]seed@VM:~$ id webapp
uid=1001(webapp) gid=1003(webapp) groups=1003(webapp),1001(appteam)
[02/16/26]seed@VM:~$ id dataengine
uid=1002(dataengine) gid=1004(dataengine) groups=1004(dataengine),1001(appteam)
[02/16/26]seed@VM:~$ id deployer
uid=1003(deployer) gid=1005(deployer) groups=1005(deployer),1002(ops)
[02/16/26]seed@VM:~$

```

Hint: Use `sudo useradd`, `sudo groupadd`, and `sudo usermod -aG`.

Task 4.2 — File Creation and Initial Ownership (5 points)

Create a project directory at `/opt/novatech/` and create the three files inside it: `config.yaml`, `analytics.py`, and `deploy.sh`. Add some sample content to each file (e.g., a comment line). Set the initial ownership of all files to `root:root`. Provide the commands used.



*** (Computer crashed on me)

Task 4.3 — Permission Mapping Strategy (5 points)

Standard Linux file permissions use three permission slots: owner, group, and others. Your task is to map each file's access requirements onto this model by carefully choosing which user owns each file, which group is assigned, and what permissions each slot receives.

For each file, determine the optimal owner and group assignment that allows the required permissions to be expressed using standard chmod. Fill in the table below:

File	Owner:Group	Owner Perms	Group Perms	Others Perms
config.yaml	deployer:appteam	rw-	r--	---
analytics.py	dataengine:appteam	rwX	r--	---
deploy.sh	deployer:ops	r-X	---	---

4. For each file, write a short explanation of why you chose that specific owner and group combination.

a) Config.yaml:

- I appointed the Deployer as the owner due to the fact that it is the only process that requires configuring this file after releases, such as saving timestamps or making deployment changes. The group was assigned as appteam to enable both the WebApp and DataEngine processes to read the necessary configuration settings for their proper functioning. This arrangement makes sure that the Deployer alone has the permission to do the changes to the file while the other services can freely access the file for reading only.

b) analytics.py:

- I chose dataengine as the owner because the DataEngine developer has the responsibility for the running and maintenance of the analytics script, which makes it imperative for the owner to be a dataengine. Because this process is required to have full control over the file, it needs to have read, write, and execute permissions. Compatibility with this decision is the appteam group assigned to the WebApp, which still reads the script to learn about report types, but cannot change or execute it. The code can be modified only by DataEngine since it is done following the principle of least privilege.

c) deploy.sh:

- The main reason for selecting deployer as the owner is that it is the only one who should have the authority over the deployment scripts. Ops has been given to the group because the operations/deployment team is the sole team who should be allowed to run the script. This keeps other processes such as WebApp and DataEngine away from deployment functionality that could cause an accidental or malicious deployment.

5. Explain the role of the "others" permission slot in your design. What general principle should apply to "others" under a least-privilege policy?

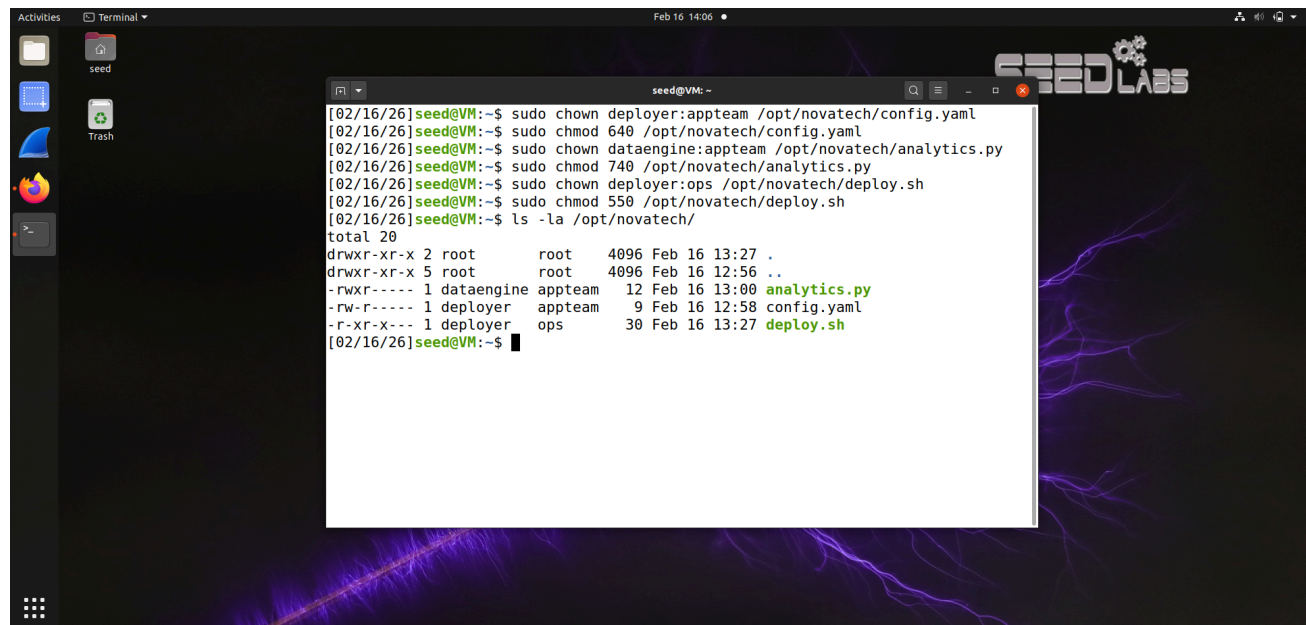
- a) In my design, the "others" permission slot means that access is granted to any user or process that is not the file owner and not part of the group assigned to the file. Following the least-privilege policy, "others" should in general have no permissions (---) unless there is a strong reason to give public access. This way, it becomes impossible to read, modify, or run system files which are sensitive and are affected by accidental exposure,

or security vulnerabilities by unauthorized users.

Task 4.4 — Implementing Permissions (5 points)

Using `chown`, `chgrp`, and `chmod`, implement the permission scheme from your table above. For each file, provide:

- The `chown` command to set the correct owner and group.
- The `chmod` command (use octal notation) to set the correct permission bits.
- Run `ls -la /opt/novatech/` after all permissions are set and include the output.



```

[02/16/26]seed@VM:~$ sudo chown deployer:appteam /opt/novatech/config.yaml
[02/16/26]seed@VM:~$ sudo chmod 640 /opt/novatech/config.yaml
[02/16/26]seed@VM:~$ sudo chown dataengine:appteam /opt/novatech/analytics.py
[02/16/26]seed@VM:~$ sudo chmod 740 /opt/novatech/analytics.py
[02/16/26]seed@VM:~$ sudo chown deployer:ops /opt/novatech/deploy.sh
[02/16/26]seed@VM:~$ sudo chmod 550 /opt/novatech/deploy.sh
[02/16/26]seed@VM:~$ ls -la /opt/novatech/
total 20
drwxr-xr-x 2 root      root      4096 Feb 16 13:27 .
drwxr-xr-x 5 root      root      4096 Feb 16 12:56 ..
-rwxr----- 1 dataengine appteam  12 Feb 16 13:00 analytics.py
-rw-r----- 1 deployer   appteam  9 Feb 16 12:58 config.yaml
-r-xr-x--- 1 deployer    ops      30 Feb 16 13:27 deploy.sh
[02/16/26]seed@VM:~$
  
```

Task 4.5 — Verification

Verify your implementation by attempting the following operations using `sudo -u username` and recording the result (success or "Permission denied"):

- As webapp, read `config.yaml` (should succeed).
- As webapp, write to `config.yaml` (should fail).
- As webapp, read `analytics.py` (should succeed).
- As dataengine, execute `analytics.py` (should succeed).
- As dataengine, read `deploy.sh` (should fail).
- As deployer, execute `deploy.sh` (should succeed).
- As deployer, read `analytics.py` (should fail).
- As deployer, write to `config.yaml` (should succeed).

```

[02/16/26]seed@VM:~$ sudo chown deployer:appteam /opt/novatech/config.yaml
[02/16/26]seed@VM:~$ sudo chmod 640 /opt/novatech/config.yaml
[02/16/26]seed@VM:~$ sudo chown dataengine:appteam /opt/novatech/analytics.py
[02/16/26]seed@VM:~$ sudo chmod 740 /opt/novatech/analytics.py
[02/16/26]seed@VM:~$ sudo chown deployer:ops /opt/novatech/deploy.sh
[02/16/26]seed@VM:~$ sudo chmod 550 /opt/novatech/deploy.sh
[02/16/26]seed@VM:~$ ls -la /opt/novatech/
total 20
drwxr-xr-x 2 root      root    4096 Feb 16 13:27 .
drwxr-xr-x 5 root      root    4096 Feb 16 12:56 ..
-rwxr----- 1 dataengine appteam 12 Feb 16 13:00 analytics.py
-rw-r----- 1 deployer  appteam  9 Feb 16 12:58 config.yaml
-r-xr-x--- 1 deployer  ops      30 Feb 16 13:27 deploy.sh
[02/16/26]seed@VM:~$ sudo -u webapp cat /opt/novatech/config.yaml
# config
[02/16/26]seed@VM:~$ sudo -u webapp sh -c 'echo test >> /opt/novatech/config.yaml'
sh: 1: cannot create /opt/novatech/config.yaml: Permission denied
[02/16/26]seed@VM:~$ sudo -u webapp cat /opt/novatech/analytics.py
# analytics
[02/16/26]seed@VM:~$ sudo -u dataengine /opt/novatech/analytics.py
[02/16/26]seed@VM:~$ sudo -u dataengine cat /opt/novatech/deploy.sh
cat: /opt/novatech/deploy.sh: Permission denied
[02/16/26]seed@VM:~$ sudo -u deployer /opt/novatech/deploy.sh
Deploying...
[02/16/26]seed@VM:~$ sudo -u deployer cat /opt/novatech/analytics.py
cat: /opt/novatech/analytics.py: Permission denied
[02/16/26]seed@VM:~$ sudo -u deployer sh -c 'echo deployed >> /opt/novatech/config.yaml'
[02/16/26]seed@VM:~$

```

Hint: Use `sudo -u username cat file` to test read, `sudo -u username sh -c 'echo test >> file'` to test write, and `sudo -u username ./file` or `sudo -u username bash file` to test execute.

Submission Guidelines

Submit a single PDF document containing all four parts. Include the completed tables (you may recreate them digitally or fill in the printed version and scan it). For Part 4, include full terminal command transcripts and screenshots of `ls -la` output and verification test results. Clearly label each part and sub-question.

Grading Rubric

Component	Points	Criteria
Part 1: Deriving Permissions	10	Correct permissions with justifications
Part 2: Access Control Matrix	10	Correct ACM + analysis paragraph
Part 3: Access Control Lists	10	Correct ACLs + comparison answers
Part 4: Linux Implementation	20	Ownership, chmod, ls -la, verification
Total	50	

Part 2 50 Points, SetUID and Environmental Variables:

The instructions for the lab can be found in either Spanish or English in the seedsecurity labs website. You will also need to download the labsetup.zip file as we saw in class.

https://seedsecuritylabs.org/Labs_20.04/Software/Environment_Variable_and_SetUID/